SessionProvider and JWT Cookies

Question

Why does the session provider store the session object instead of storing the JWT token? Why can't the app verify if there is a JWT token in the cookie directly? For example, if there is such a cookie or a token, then the user is logged in. If there is not a token, then the user is not logged in.

Answer

Because the browser/app should not treat "cookie exists" as proof of login.

Main reasons:

1. The JWT cookie is protected server-side.

In your NextAuth version, the JWT is encrypted. The browser can store it, but your React app cannot meaningfully read or verify it.

Also, the cookie is usually `HttpOnly`, meaning JavaScript cannot access it:

```
document.cookie // will not show HttpOnly auth cookie
```

That is intentional. It protects the token from client-side JavaScript and XSS attacks.

2. Cookie exists does not mean cookie is valid.

A cookie might be:

```
expired
tampered with
encrypted with an old secret
malformed
for a deleted user
for a user whose permissions changed
```

Only the server has the secret and trusted logic to verify it.

3. The app needs a safe user object, not the raw token.

Frontend components usually need:

```
session.user.id
session.user.email
session.user.name
```

They do not need the encrypted JWT string.

So NextAuth converts:

```
cookie token
-> verified/decrypted server-side payload
-> safe session object
-> frontend
```

4. Security boundary.

The client is not trusted to decide authentication by itself. The server must decide:

```
Is this request authenticated?
```

That is why `/api/auth/session` exists.

Correct mental model:

```
JWT cookie = private proof stored by browser, verified by server
session object = safe login state exposed to React app
```

So SessionProvider stores the session object because that is the useful, safe, already-verified result. It does not store the JWT because the client should not need it and usually cannot read it.